Smart MCQ Solver Challenge - Project Report

Course: BSDA2001P - Introduction to Deep Learning and Generative AI

Name: Ashvin Ganeshrao Ambatwar

Email Id: 24f2009283@ds.study.iitm.ac.in

Kaggle competition: smart-mcq-solver-challenge

Repository: github.com/ashwinambatwariitm/dlgenai-t22026 (private)

Deployed model: huggingface.co/ashwinambatwariitm/dlgenai-t22026-mcq

Live demo: https://smart-mcq-solver-dlgenai.streamlit.app/

---

1. Abstract / Executive Summary

Problem. Rank five answer choices for 500 test questions and submit the top three, scored by MAP@3.

Methods. Multiple approaches were evaluated, including TF-IDF, MiniLM, from-scratch neural models, fine-tuned BERT, RAG and a weighted Borda ensemble.

Results. The ensemble achieved the best validation MAP@3 (**0.8858**), followed by BERT (**0.8729**) and BiLSTM (**0.8633**). The best Kaggle score was **0.75664**, above the **0.73** cutoff.

Main learning. The answer-text artifact baseline scored **0.7954** without using the question, showing strong dataset bias. Also, the cluster-based validation split removes near-duplicates, while the Kaggle test set contains many repeated training questions. Thus, validation mainly measures generalization, whereas leaderboard performance is also affected by memorization.

---

2. Introduction

2.1 Problem Statement

Given a prompt and five options A to E, rank the top three answers. The dataset has 2,000 labelled training and 500 test questions. MAP@3 gives 1.0, 0.5, or 1/3 when the correct answer is ranked 1st, 2nd, or 3rd.

2.2 Project Objective

1. Exceed the **0.73 qualifying cutoff**.

2. Compare pretrained, custom, and sequence models under the same evaluation protocol.
3. Gain practical experience with **PyTorch, Hugging Face, W&B, and deployment**.
4. Build a reliable evaluation setup and identify dataset biases or leakage.

2.3 Report Structure

Section 3 covers the dataset and leakage, Section 4 tokenisation, Section 5 models, Section 6 results and validation-leaderboard differences, and Section 7 conclusions.

---

3. Dataset and Preprocessing

3.1 Dataset Description

The dataset contains **2,000 labelled training and 500 test questions**, each with a prompt and five options A to E. There are no missing values and every question has exactly five options. Topics include philosophy, physics and general academic knowledge.

3.2 Exploratory Data Analysis

The answer labels are mildly imbalanced, with B and C occurring most often. The constant **B C D** baseline achieves **0.4088 MAP@3**, compared with **0.3937** for A B C.

Prompts are short, averaging **18.1 words** (maximum 51), while the longest option per question averages **30.5 words** (maximum 118). These statistics guided the sequence-length and padding choices.

Finding 1: High duplication. Clustering prompts using embedding cosine similarity of 0.95 reduces the 2,000 rows to **613 distinct question clusters**. Therefore, the effective diversity of the dataset is much smaller than its row count.

Finding 2: Answer-text artifact. There are **313 distinct correct-answer texts** and **1,271 distractor texts**, with no overlap. Thus, some questions can be answered from the option text alone. This artifact baseline achieves **0.7954 MAP@3** without reading the question.

3.3 Data Preprocessing

Because of the duplication, a random validation split would cause severe leakage. The final split therefore groups near-duplicate questions into clusters before splitting.

split	validation questions with a near-duplicate in train
random 80/20	79.5%
cluster-based (used)	**0.0%**

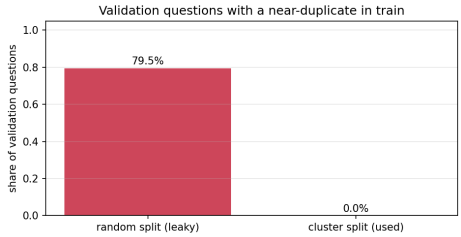


Figure: `fig_leakage_before_after.png`

Using all-MiniLM-L6-v2 with a cosine threshold of 0.95 produced **613 clusters**. Whole clusters were assigned to train or validation, giving **1,600 / 400 questions** with no question appearing on both sides.

The final training distribution was checked after splitting to ensure that the answer-label imbalance remained acceptable.

---

4. Tokenisation Strategy

4.1 Primary Tokeniser

WordPiece with `bert-base-uncased` uses a 30,522-token vocabulary and splits rare words into familiar subwords. Each option is encoded together with the prompt as a **sentence pair**, allowing BERT to distinguish the two parts.

4.2 Implementation Details

Across 10,000 prompt-option pairs, token lengths were **56 tokens at p50, 91 at p90, 131 at p99, and 200 at maximum**. Therefore, `max_length=256` caused no truncation.

Instead of padding every sequence to 256 tokens, batches were padded only to their own longest sequence. This reduced the mean padded length from **192 to 107 tokens**, about **1.8x less computation per step**. The resulting input shape is `(batch, 5, seq_len)`, as required by `AutoModelForMultipleChoice`.

4.3 Experimental Comparison

WordPiece was used for BERT because it is the tokenizer matched to the pretrained bert-base-uncased vocabulary and weights. For the from-scratch models, a simple whitespace and regex tokenizer was used. Dynamic padding was applied to reduce unnecessary computation.

WordPiece was used for BERT because it is the tokenizer matched to the pretrained bert-base-uncased vocabulary and weights.

5. Modelling and Experimentation

All models were evaluated on the same leakage-free split using the same MAP@3 implementation.

5.1 Model 1: Fine-Tuned Transformer

`bert-base-uncased` with a multiple-choice head was fine-tuned. It has **12 layers, 768 hidden units, 12 attention heads and about 110M parameters**.

Each option is paired with the prompt and scored independently by the shared BERT encoder. The five scores are then ranked to select the answers.

Setting	Value
Fine-tuning	Full
Optimizer	AdamW, lr = 2e-5
Batch	16 Effective
Max Length	256, dynamic padding
Precision	FP16
Epochs	10, best epoch=6

An initial LoRA experiment was under-trained, so full fine-tuning was used. Validation MAP@3 reached **0.8729 at epoch 6**, after which performance remained almost unchanged while training loss continued to fall. This indicates the model had started to overfit. Top-1 accuracy was **0.7925**.

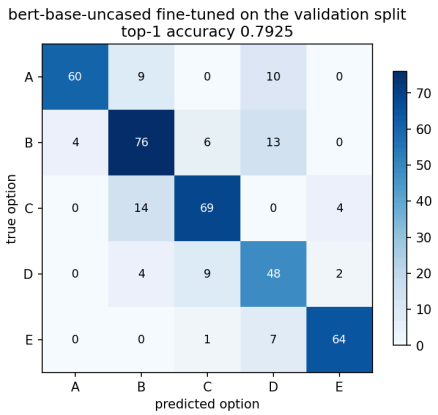


Figure: `fig_confusion_matrix.png`

5.2 Model 2: Custom Deep Learning Model

`ScratchMCQModel` was built entirely from scratch, using its own vocabulary, randomly initialized embeddings and a **shared encoder** for the prompt and each option.

The scorer combines the prompt and option representations using `[p; o; |p-o|; p*o]`, followed by a small MLP to produce one score for each option. **Masked mean pooling** prevents padding tokens from affecting the representation.

The model was trained with Adam (lr = 1e-3) for 8 epochs.

Encoder ablation. The same scoring head and training setup were used with three different encoders:

Encoder	MAP@3
BiLSTM	**0.8633**
text-CNN	0.8462
mini transformer	0.8471

The BiLSTM performed best among the from-scratch models. The mini Transformer also provides a useful comparison with BERT: **0.8471 from scratch vs 0.8729 after fine-tuning**, showing the benefit of adapting pretrained weights to the task.

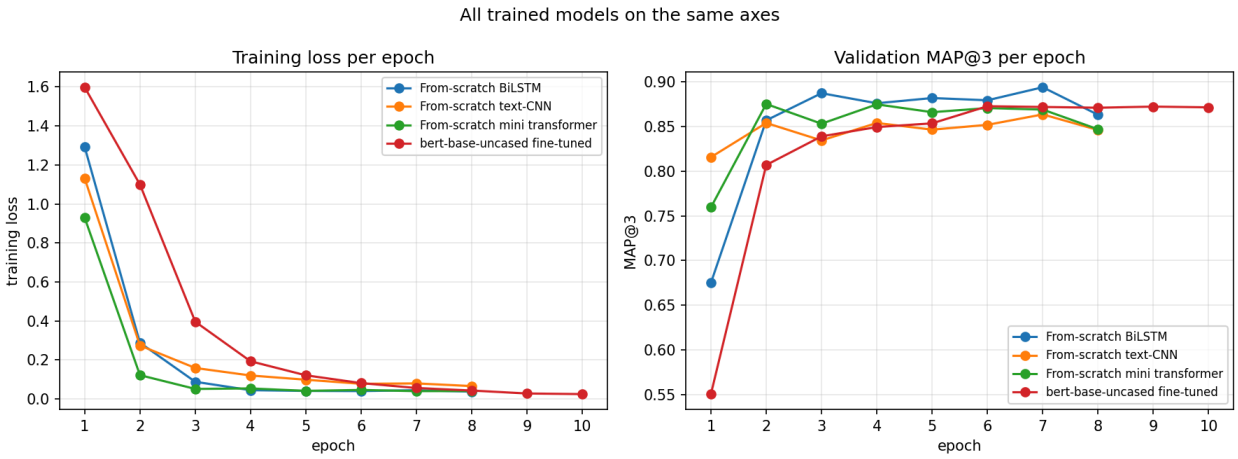


Figure: `fig_learning_curves_all_models.png`.

5.3 Additional Experiments

system	Approach	MAP@3
TF-IDF + LogReg / SVM	1-2 gram features	0.7733/0.7596

system	Approach	MAP@3
MiniLM zero-shot	Sentence embeddings + cosine similarity	0.4846
RAG + Qwen2.5-0.5B-Instruct	Retrieve 3 solved questions and score options	0.8208
Weighted Borda ensemble	Rank fusion over TF-IDF, BiLSTM, BERT and RAG	0.8858

Borda fusion was used because the models produce scores on different scales. Combining their **rankings** avoids the need to calibrate these scores.

---

6. Performance and Comparative Analysis

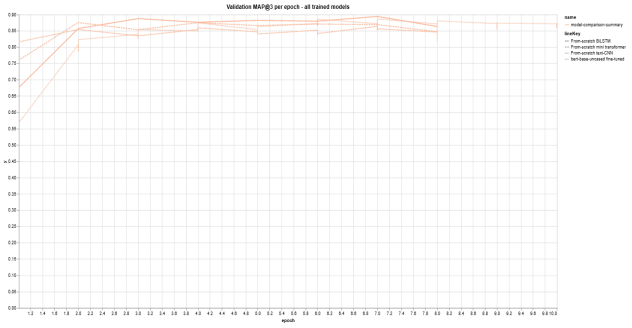
6.1 Evaluation Metrics

MAP@3 is the main competition metric. With one correct answer, it gives **1.0, 0.5, or 1/3** when the correct option is ranked 1st, 2nd, or 3rd, and 0 otherwise. The implementation was checked against the competition example.

6.2 Training Performance

The training curves show that BERT's training loss decreases to **0.0241**, while validation performance stops improving after epoch 6. Therefore, the best validation epoch was selected instead of using the final epoch.





Experiment tracking was done using **Weights & Biases (W&B)**, recording training curves, model comparison results, confusion matrix, leakage analysis and submission artifacts. wandb.ai/ambatwar-iitm/dlgenai-t226.

6.3 Comparative Report

#	model	family	MAP@3
1	**Ensemble (weighted Borda)**	fusion	**0.8858**

#	model	family	MAP@3
2	`bert-base-uncased` fine-tuned	pretrained transformer	0.8729
3	From-scratch BiLSTM	custom / RNN	0.8633
4	From-scratch mini transformer	custom / self-attention	0.8471
5	From-scratch text-CNN	custom / convolutional	0.8462
6	RAG + Qwen2.5-0.5B	retrieval + causal LM	0.8208
7	**Answer-text artifact**	**baseline, reads no question**	**0.7954**
8	TF-IDF + Logistic Regression	classical ML	0.7733
9	TF-IDF + linear SVM	classical ML	0.7596
10	MiniLM (zero-shot)	pretrained, frozen	0.4846
11	Constant `B C D`	baseline	0.4088
12	Constant `A B C`	baseline	0.3937

The answer-text artifact is not a model. It predicts using only whether an option's text appeared as a correct answer during training, without reading the question. It is included because it reveals a strong dataset bias.

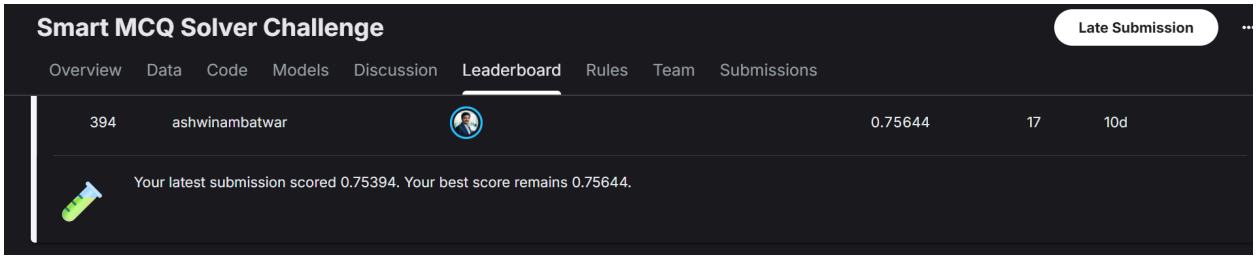
Therefore, **0.7954 is the more meaningful baseline**, rather than the 0.3937 constant baseline. Compared with this baseline, BERT improves by **0.078**, while the ensemble improves by **0.090**.

Key observations.

1. **Pretraining helps after fine-tuning:** MiniLM zero-shot scores 0.4846, while fine-tuned BERT reaches 0.8729.
2. **Sequence models outperform TF-IDF:** BiLSTM, CNN and Transformer all score higher than both TF-IDF approaches.
3. **BiLSTM performed strongly from scratch:** It reached 0.8633 MAP@3, close to the fine-tuned BERT score of 0.8729.
4. **The ensemble performs best:** 0.8858, only 0.013 above BERT. This difference is small compared with the observed validation variation, so it should not be treated as a definitive advantage.

6.4 Kaggle Performance and the Validation Gap

The best Kaggle score was **0.75664 MAP@3 (rank 394)**, above the **0.73 qualifying cutoff**. The final notebook run scored 0.75394.



submitted system	validation MAP@3	Kaggle MAP@3
Ensemble + retrieval, run A	0.8742	**0.75664**
Ensemble + retrieval, run B (final)	**0.8858**	0.75394
RAG + Qwen2.5-0.5B	0.8208	0.75436
bert-base-uncased, 5 epochs	0.8371	0.75228
bert-base-uncased, 8 epochs	0.8829	0.74937

The validation scores range from 0.82 to 0.89, while Kaggle scores are much closer together. Two identical runs also showed opposite movement: validation increased by **0.0116**, while Kaggle decreased by **0.0027**.

The main reason is the composition of the test set. Exact matching against the training data showed:

Relationship to train

	Test share
Identical prompt and options	**52.8%** (264/500)
Same prompt, different options	0.8% (4/500)
different prompt, same option set	38.8% (194/500)
no match at all	7.6% (38/500)

The cluster-based validation set mainly measures generalisation, while the leaderboard also rewards recognising questions already present in training.

The hybrid retrieval layer confirms this effect. It retrieved the correct training answer for **466 of 500 test questions (93.2%)**, using the nearest training question and matching its

correct option text. This is better described as **memorisation through retrieval**, rather than improved reasoning.

6.5 Error Analysis

Validation leakage. The original random split had **79.5% near-duplicate overlap**, producing unrealistically high scores. Cluster-based splitting reduced this to **0%**. For example, TF-IDF + SVM fell from 0.9721 to 0.7596 after removing this leakage.

Answer-text artifact. Correct-answer texts and distractor texts had no overlap. The resulting question-blind baseline achieved **0.7954 MAP@3**, showing that the option text itself contains strong predictive information.

Over-training. BERT at 8 epochs improved validation from 0.8371 to 0.8829 but reduced Kaggle performance from 0.75228 to 0.74937. This shows that a higher validation score does not necessarily mean better leaderboard performance.

Run-to-run variation. Repeated runs changed validation by about **0.01** and Kaggle by about **0.003**. Therefore, small differences between models should not be treated as definitive. The clearest differences are the consistently higher performance of neural sequence models over TF-IDF and the large improvement from fine-tuning compared with zero-shot MiniLM.

---

7. Conclusion and Future Work

7.1 Key Learnings and challenges

1. **Evaluation matters as much as the model.** Detecting duplicate questions and creating a leakage-free split made the model comparison more reliable.
2. **Dataset artifacts strongly affect performance.** The question-blind answer-text baseline achieved **0.7954 MAP@3**, showing that option text itself contains useful information.
3. **Fine-tuning is important.** Zero-shot MiniLM scored 0.4846, while fine-tuned BERT reached 0.8729. A from-scratch Transformer reached 0.8471, showing that both architecture and task adaptation contribute.
4. **Different models provide complementary information.** Combining their rankings with weighted Borda fusion produced the best validation score of **0.8858**.
5. **Small score differences should be interpreted carefully.** Repeated runs showed approximately 0.01 validation variation, so closely spaced results are not necessarily significant.

7.2 Challenges Faced

The main challenges were handling **data leakage, GPU memory and compute limits**. Cluster-based splitting reduced near-duplicate validation leakage to 0%. Dynamic padding reduced unnecessary computation, while models were freed from GPU memory after use.

Another important challenge was the difference between embedding-based similarity and exact matching. An early similarity estimate suggested much higher duplication than the stricter exact-match analysis, demonstrating the importance of validating assumptions with the actual data.

7.3 Future Work

1. Use **cluster-based cross-validation** to obtain more reliable performance estimates.
2. Train on larger external multiple-choice datasets such as RACE, ARC or OpenBookQA to improve generalisation.
3. Explore stronger retrieval and reranking methods while separating memorisation from genuine question understanding.

7.4 Deployment

The fine-tuned BERT model is published at
huggingface.co/ashwinambatwariitm/dlgenai-t22026-mcq

A Streamlit application was also prepared to accept a question and five options and return the predicted top-3 answers.

---

8. References

1. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL-HLT. arXiv:1810.04805
2. Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* arXiv:1907.11692
3. He, P., Gao, J., Chen, W. (2023). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.* ICLR. arXiv:2111.09543
4. Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP. arXiv:1908.10084
5. Conneau, A. et al. (2017). *Supervised Learning of Universal Sentence Representations from Natural Language Inference Data.* EMNLP. arXiv:1705.02364
6. Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685
7. Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR.